the primary controller. An INA219 current/voltage sensor is interfaced via the I²C bus to monitor the electrical parameters of the servo monitor grips. Four servo motors are driven by a PCA9685 16-channel, 12-bit PWM driver, which provides multiple independent outputs for precise servo positioning. The PCA9685 communicates with the ESP32 via I²C, allowing the ESP32 to send PWM commands to control the angle of each servo. A 0.96-inch (2.4cm) SSD1306 128 × 64 OLED display is also connected to the ESP32 via I²C to show voltage, current, and other operating information. Capacitors C1 and C2 provide local supply decoupling to reduce noise and ensure stable operation. The system is powered through an ATX24R connector, which provides the necessary supply rails and a common ground. During operation, the ESP32 processes control commands, drives the PCA9685 to operate the four servo motors, and simultaneously receives voltage/current data from the INA219. The measured parameters are displayed on the OLED, providing real-time information about the servo system.

The robotic shopkeeper uses a hybrid control architecture in which a laptop, Raspberry Pi 5, or GPU server acts as the high-level processing unit, while the ESP32 handles real-time hardware control. The server performs object detection, AI-based decision-making, path planning, and high-level control, while the ESP32 handles real-time interfacing with motors, sensors, encoders, and the gripper. Communication between the server and the ESP32 is performed wirelessly through ESP-NOW, reducing cable clutter.

The robotic arm uses four servo motors, three of which provide the arm's DoF, while the fourth operates the gripper. These are controlled through a PCA9685 I²C servo driver, which supports up to 16 servos. An INA219 current sensor monitors the gripper servo's current. When the gripper achieves the required gripping force, the servo stops at a predefined current threshold, preventing excessive force and protecting the servo gears. This is particularly important when handling delicate objects, such as vegetables, where controlled gripping pressure is required. A VL53L0X time-of-flight distance sensor measures the distance between the gripper and the target, triggering the gripping action when the object is within the required range. The PCA9685, VL53L0X, OLED, and INA219 sensors all use the same I²C bus, sharing the same pins and wiring with the microcontroller.

The ESP32 firmware is largely self-contained and continuously collects sensor data, controls the hardware, and communicates with the server. The Python application running on the server acts as the decision-making layer, processing sensor and camera information and sending appropriate commands to the ESP32.

Visual and loop control

For object detection, YOLO-E was selected due to its compact size and ability to detect custom objects, such as matchboxes or pencil sharpeners. However, other YOLO models are also compatible. Proper lighting, camera positioning, and calibration are essential for reliable detection. The camera is positioned along Servo0's rotational axis, so the detected visual X-Y coordinates align closely with the robotic arm's working coordinates, improving the accuracy of initial target positioning.

The system utilises a semi-closed-loop approach. The camera initially detects the object and provides its X-Y position to the robotic arm. The arm then moves towards the target, while the VL53L0X sensor provides distance feedback during the final approach. Because the camera is stationary rather than mounted on the robotic arm, continuous vision-based closed-loop tracking is not possible.

Furthermore, as the arm moves, its structure can obstruct the camera's view of the target. Therefore, the camera is primarily used for initial object detection and positioning, while the VL53L0X sensor provides final-approach feedback. Except for the load cell, all other sensors in the system have been integrated and are functioning as intended. The gripper function is currently under development.

Coding and calibration

Two main programs are required for the robotic system. The first is the Python-based object-detection program, which can run on a Raspberry Pi 5. However, a laptop or GPU-equipped computer is preferred when higher processing speed and frame rates are required. The necessary Python libraries and modules must be installed before running the detection program to ensure accurate and reliable operation. The second is the ESP32 control firmware, which receives commands from the Python application and controls the robotic arm, sensors, and servo motors.

Calibration

Calibration is required to transform the camera coordinates into the robotic arm's coordinate system. Since the camera is mounted close to the rotational axis of Servo0, its X-Y coordinate system approximately corresponds to the arm's actual X-Y workspace. However, the camera cannot be positioned exactly on the Servo0 axis, resulting in a small positional offset. Calibration compensates for this offset and improves the positioning accuracy of the robotic arm. Two calibration procedures are required:

1. Servo0/Base table calibration – basetable_alignment.py

This establishes the relationship between the camera coordinates and the Servo0 base rotation.

2. Servo1 and Servo2/Arm reach calibration – reach_alignment.py

This calibrates the relationship between the target position and the reach of Servo1 and Servo2.

Both calibration programs are menu-driven and provide the required